

EC8208 – Big Data Analytics

Continuous Assessment 02

Take Home Assignment

Kafka Avro Order Processing System

Name : Threemavithana T.M.

Reg. No : EG/2021/4835

Repository: github.com/TM-threemavithana/kafka-avro-order-processing-system

Executive summary

This project implements the complete event-processing flow required by the assignment. Orders are serialized with Apache Avro and published to Kafka. A consumer validates each record, calculates both overall and per-product running price averages, and persists aggregate state so that the results survive a restart. Temporary failures are retried with an exponential delay, while permanent, invalid, or retry-exhausted records are preserved in a dead-letter queue (DLQ).

The repository also provides Docker-based infrastructure, a deterministic demonstration, automated unit tests, Kafka integration tests, and continuous integration. The latest verified test run contains 39 passing tests. The demo publishes ten orders: nine are ultimately processed successfully and one permanent failure is routed to the DLQ.

1 Project objectives

- Create a reproducible local Kafka environment for producing and consuming order events.
- Enforce a clear Avro contract with the exact assignment fields and compatible types.
- Compute and display real-time overall and per-product running price averages.
- Separate recoverable failures from permanent failures without losing the original record.
- Provide automated evidence that the normal, retry, DLQ, and persistence paths work correctly.

2 System design

The design uses three Kafka topics. The main topic carries new orders, the retry topic holds temporary failures until they become eligible for another attempt, and the DLQ retains records that cannot be processed successfully. This separation keeps normal processing responsive while making failure handling visible and auditable.

Kafka order-processing data flow

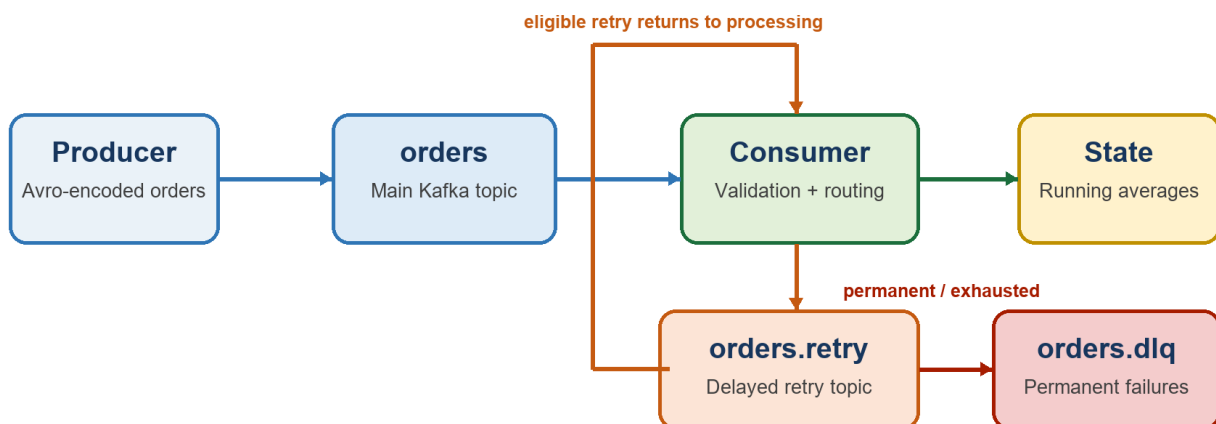


Figure 1. End-to-end Kafka order-processing architecture

2.1 Components and responsibilities

Component	Responsibility
Order producer	Validates order input, serializes it with Avro, and publishes to orders.
Order consumer	Deserializes, validates, applies failure policy, updates aggregates, and commits offsets.
Retry path	Adds retry headers and a not-before timestamp, then reprocesses eligible records.
DLQ path	Preserves failed payloads and diagnostic headers for inspection.
State store	Persists per-product count, total, average inputs, and duplicate-detection data.
Docker environment	Starts Kafka and supporting services consistently for development and demonstration.

2.2 Kafka topics

Topic	Purpose	Produced by	Consumed by
orders	New order events	Order producer	Order consumer
orders.retry	Recoverable failures awaiting retry	Order consumer	Order consumer
orders.dlq	Permanent, invalid, or exhausted records	Order consumer	DLQ viewer / operator

3 Avro message contract

The schema defines an Order record in the `lk.assignment.orders` namespace. The three fields match the assignment exactly. Validation additionally rejects missing identifiers, blank product names, non-finite numbers, negative prices, and values outside the Avro float range.

Field	Avro type	Meaning	Validation
orderId	string	Unique order identifier	Required and non-blank
product	string	Product name or key	Required and non-blank
price	float	Order price	Finite, non-negative, Avro-float compatible

3.1 Schema definition

```
{
  "type": "record",
  "name": "Order",
  "namespace": "lk.assignment.orders",
  "fields": [
    {"name": "orderId", "type": "string"},
    {"name": "product", "type": "string"},
    {"name": "price", "type": "float"}
  ]
}
```

3.2 Serialization flow

1. The producer validates the Python order object against application rules.
2. The object is serialized using the Avro schema before it is sent to Kafka.
3. The consumer deserializes the payload with the same schema and performs defensive validation.
4. A valid order enters aggregation; a malformed or invalid order is routed to the DLQ with context.

4 Processing and aggregation

For every successfully processed order, the consumer updates an aggregate keyed by product. If n is the number of accepted orders for a product and S is their accumulated price, the running average is calculated as S / n . Maintaining count and total avoids rescanning prior messages. The system calculates two types of running averages. The overall average represents all successfully processed orders, while each per-product average includes only orders belonging to that product.

$$\text{average}(\text{product}) = \text{total price}(\text{product}) / \text{accepted order count}(\text{product})$$

$$\text{Overall average} = \text{total price of all accepted orders} / \text{total number of accepted orders}$$

$$\text{Per-product average} = \text{total accepted price for the product} / \text{accepted order count for the product}$$

4.1 State, duplicates, and consistency

- Aggregate state is written to a local state file and loaded when the consumer restarts.
- Processed order identifiers are retained to prevent a redelivered record from changing the average twice.
- A duplicate with the same data is ignored safely; a conflicting duplicate is identified as an error.
- Kafka offsets are committed manually only after processing succeeds or the record is safely rerouted.

4.2 Delivery settings

Control	Implementation	Benefit
Producer idempotence	Enabled	Reduces duplicate records caused by producer retries
Acknowledgements	acks=all	Waits for the strongest configured broker acknowledgement
Consumer commits	Manual	Avoids acknowledging a record before processing or safe rerouting
Duplicate detection	orderId-based	Protects aggregate results during redelivery

5 Failure handling

Failures are classified before the consumer commits the source offset. Recoverable errors are published to the retry topic; permanent or invalid records are sent directly to the DLQ. A temporary record that reaches the retry limit is also sent to the DLQ, ensuring every terminal outcome is visible.

Failure category	Example	System action	Terminal result
Temporary	Simulated transient service error	Retry with attempt count, reason, and not-before timestamp	Success or DLQ after limit
Permanent	Business rule marks order as unrecoverable	Route immediately with failure metadata	DLQ
Invalid data	Malformed Avro or invalid field value	Preserve payload and validation context	DLQ
Conflicting duplicate	Same orderId with different order data	Reject conflicting replay	DLQ

5.1 Temporary retry behavior

The demonstration policy intentionally fails a designated temporary order on attempts 0 and 1 and succeeds on attempt 2. Each retry carries its attempt number and an eligibility time. The consumer does not block while waiting; it continues handling records and revisits the retry when its not-before time has passed. Exponential backoff spaces repeated attempts and prevents a tight retry loop.

5.2 DLQ observability

DLQ records retain the original message plus diagnostic headers such as the failure reason and retry context. The supplied DLQ viewer allows an assessor or operator to inspect failed orders without altering the main consumer state.

6 Verification and evidence

The project combines focused unit tests with a Docker-backed Kafka integration workflow. The latest verified suite contains 39 passing tests, and the repository's GitHub Actions workflow completes both the unit-test and Kafka-integration jobs successfully.

Verification area	Evidence checked	Result
Avro serialization	Valid round trips and malformed payload behavior	Pass
Input validation	Required text, finite price, non-negative price, float range	Pass
Aggregation	Count, total, average, persistence, duplicate handling	Pass
Retry policy	Headers, attempt sequence, backoff, not-before scheduling	Pass
DLQ routing	Permanent, invalid, and exhausted failure paths	Pass
Demo batch	Expected successful count and aggregate outcomes	Pass
Kafka integration	Real topics, retry recovery, and permanent DLQ delivery	Pass

6.1 Deterministic demonstration result

The producer generates randomized order prices. For the demonstration, seed 42 is used to generate the same reproducible ten-order batch on every clean execution.

Metric	Expected / verified value
Orders published	10
Normal orders	8
Temporary-failure order	1; succeeds on retry attempt 2
Permanent-failure order	1; delivered to orders.dlq
Orders included in final aggregates	9
Final overall average	197.86
Automated tests	39 passing
CI evidence	GitHub Actions run, unit-test and Kafka-integration jobs completed successfully

7 Live demonstration guide

The following sequence presents the required behavior clearly. Run the commands from the repository root in separate terminals where indicated. Exact command names are also documented in the README.

1. Start the Docker services and confirm that Kafka is healthy.
2. Create or verify the orders, orders.retry, and orders.dlq topics.
3. Start the order consumer so that processing output is visible.
4. Run the deterministic producer/demo batch containing normal, temporary, and permanent cases.
5. Show that the temporary order is retried twice and then included in the aggregates.
6. Open the DLQ viewer and show the single permanent-failure record with its diagnostic reason.
7. Show the final successful aggregate count of nine and run the automated test suite.

8 Conclusion

The Kafka Avro Order Processing System satisfies the stated assignment requirements end to end. It produces and consumes schema-controlled orders, calculates persistent running averages, recovers temporary failures through delayed retries, and isolates permanent failures in a DLQ. Docker configuration, a deterministic live demo, comprehensive automated tests, CI, and repository documentation make the result reproducible and ready for assessment.

9 References

- [1] Apache Software Foundation, “Apache Kafka documentation.” [Online]. Available: <https://kafka.apache.org/documentation/>.
- [2] Apache Software Foundation, “Apache Kafka downloads.” [Online]. Available: <https://kafka.apache.org/downloads/>.
- [3] Confluent, Inc., “Python client for Apache Kafka,” *Confluent Documentation*. [Online]. Available: <https://docs.confluent.io/kafka-clients/python/current/overview.html>.
- [4] Apache Software Foundation, “Apache Avro specification.” [Online]. Available: <https://avro.apache.org/docs/current/specification/>.
- [5] fastavro contributors, “fastavro documentation.” [Online]. Available: <https://fastavro.readthedocs.io/en/latest/>.
- [6] Docker, Inc., “Docker Compose,” *Docker Documentation*. [Online]. Available: <https://docs.docker.com/compose/>.
- [7] T. M. Threemavithana, “Kafka Avro Order Processing System,” GitHub repository, 2026. [Online]. Available: <https://github.com/TM-threemavithana/kafka-avro-order-processing-system>.